

Pi-CloudDOOM

Hybrid Deployment Guide

OpenStack Kubernetes + Azure Container Apps + Vercel + CI/CD

Branch baseline: integration/final

Target branch: deployment/hybrid-openstack-azure-vercel

Purpose

This document is a detailed execution guide for deploying the final integrated Pi-CloudDOOM microservice application while satisfying the Phase 5 validation grid. It assumes the OpenStack Kubernetes cluster already exists and that the project Dockerfiles are working.

Table of Contents

1. 1. Deployment strategy and grading-grid alignment
2. 2. Final hybrid architecture
3. 3. Repository and branch preparation
4. 4. OpenStack Kubernetes preparation
5. 5. Kubernetes manifests and infrastructure services
6. 6. Backend microservices deployment
7. 7. Azure deployment for AI services
8. 8. Vercel deployment for Angular frontend
9. 9. Ingress and external access
10. 10. CI/CD with GitHub Actions and Azure Container Registry
11. 11. Observability, HPA, and resource optimization
12. 12. Final validation checklist and screenshots
13. 13. Troubleshooting guide
14. 14. Report-ready justification text

1. Deployment strategy and grading-grid alignment

The Phase 5 validation grid evaluates four major areas: containerization, Kubernetes deployment, infrastructure/access, and excellence/bonus. The recommended plan is not to deploy everything on OpenStack, because the OpenStack resources are limited. Instead, the deployment should use OpenStack for the core backend and infrastructure, Vercel for the static frontend, and Azure for heavy or variable-load AI services.

Grid category	Target evidence	Deployment choice
1. Containerization	Separate Dockerfiles, separate images, registry push, inter-container communication	Build every service image and push to Azure Container Registry. Keep PostgreSQL, Redis, Kafka, Keycloak, and MinIO as containers in Kubernetes.
2. Kubernetes Deployment	Deployment, Service, ConfigMap, Secret, functional pods, external access, HPA	Deploy core backends and infrastructure services on OpenStack Kubernetes. Use Ingress, ConfigMaps, Secrets, resource limits, and HPA.
3. Infrastructure & Access	OpenStack cluster, accessible app, stable infrastructure access	Use the existing OpenStack Kubernetes cluster as the main backend platform. Expose API and Keycloak through Ingress.
4. Excellence / Bonus	Hybrid deployment, CI/CD, resource limits, observability	Use Azure Container Apps for AI, Vercel for frontend, GitHub Actions for CI/CD, Prometheus/Grafana for monitoring, and Argo CD for GitOps.

Main decision

Deploy the core microservices on OpenStack Kubernetes, but move the Angular frontend to Vercel and AI-heavy services to Azure Container Apps. This protects limited OpenStack CPU/RAM while still proving Kubernetes deployment, cloud integration, CI/CD, and observability.

2. Final hybrid architecture

The target architecture separates responsibilities by runtime characteristics. Stateful and core backend components stay in OpenStack Kubernetes. Static frontend hosting is offloaded to Vercel. AI workloads are offloaded to Azure Container Apps.

GitHub Repository: Mohamed-Fedi-Hamrouni/Pi-CloudDOOM

Branch: integration/final

Deployment branch: deployment/hybrid-openstack-azure-vercel

GitHub Actions

-> Build service images

-> Push images to Azure Container Registry

Azure Container Registry

-> Stores backend, frontend, and AI images

OpenStack Kubernetes

-> PostgreSQL
-> Redis
-> Kafka + Zookeeper
-> Keycloak
-> MinIO
-> user-service
-> interview-service
-> training-service
-> quiz-service
-> community-service
-> resource-service
-> mentorship-service
-> Ingress Controller
-> Prometheus + Grafana
-> Argo CD

Azure Container Apps

-> ai-training-path
-> optional kokoro / whisper services

Vercel

-> Angular frontend

Component	Target platform	Reason
Angular frontend	Vercel	Static production build. Saves OpenStack resources and gives simple GitHub-based frontend deployment.
Core backend microservices	OpenStack Kubernetes	Required project target. Demonstrates Kubernetes Deployments, Services, ConfigMaps, Secrets, Ingress, and pod communication.
PostgreSQL, Redis, Kafka, Keycloak, MinIO	OpenStack Kubernetes	Needed for internal microservice communication and local infrastructure demonstration.
AI services	Azure Container Apps	AI workloads can be CPU/RAM-heavy and variable. Azure Container Apps supports autoscaling and scale-to-zero scenarios.
Container images	Azure Container Registry	Central private registry for

		images used by OpenStack Kubernetes and Azure Container Apps.
--	--	---

3. Repository and branch preparation

Start from the final integrated application branch, not from main or old deployment branches.

```
git clone -b integration/final https://github.com/Mohamed-Fedi-Hamrouni/Pi-CloudDOOM.git
cd Pi-CloudDOOM
git checkout -b deployment/hybrid-openstack-azure-vercel
```

Create the deployment folder structure:

```
mkdir -p k8s/namespaces k8s/config k8s/secrets
mkdir -p k8s/infra/postgres k8s/infra/redis k8s/infra/kafka k8s/infra/keycloak k8s/infra/minio
mkdir -p k8s/apps/user-service k8s/apps/interview-service k8s/apps/training-service
mkdir -p k8s/apps/quiz-service k8s/apps/community-service k8s/apps/resource-service k8s/apps/mentorship-service
mkdir -p k8s/ingress k8s/monitoring k8s/argocd
mkdir -p azure/container-apps vercel .github/workflows
```

4. OpenStack Kubernetes preparation

Because the infrastructure is resource-limited, prefer 1 control-plane node and 4 worker nodes over 3 control-plane nodes and 2 workers. The project grade depends on successful workload deployment more than control-plane high availability.

Node	Role	Recommended workload
cp-1	control-plane	Kubernetes API server, scheduler, controller manager only.
worker-1	worker	user-service, interview-service, training-service.
worker-2	worker	quiz-service, community-service.
worker-3	worker	PostgreSQL, Redis, Kafka, Keycloak.
worker-4	worker	resource-service, mentorship-service, MinIO, monitoring, Argo CD.

```
kubectl get nodes -o wide
kubectl get pods -A
kubectl get storageclass
```

Storage warning

If no StorageClass exists, persistent components such as PostgreSQL and MinIO will not be able to create PersistentVolumeClaims. For a demo, local-path-provisioner is acceptable. For a more professional OpenStack deployment, use Cinder CSI.

```
# Demo fallback if no StorageClass exists
```

```
kubectl apply -f https://raw.githubusercontent.com/rancher/local-path-provisioner/master/deploy/local-path-storage.yaml
```

```
kubectl get storageclass
```

5. Kubernetes manifests and infrastructure services

5.1 Namespace

```
# k8s/namespaces/picloud doom.yaml
```

```
apiVersion: v1
```

```
kind: Namespace
```

```
metadata:
```

```
  name: picloud doom
```

```
kubectl apply -f k8s/namespaces/picloud doom.yaml
```

5.2 ResourceQuota

Add a ResourceQuota to show resource governance and avoid exhausting the OpenStack cluster.

```
# k8s/namespaces/resource-quota.yaml
```

```
apiVersion: v1
```

```
kind: ResourceQuota
```

```
metadata:
```

```
  name: picloud doom-quota
```

```
  namespace: picloud doom
```

```
spec:
```

```
  hard:
```

```
    requests.cpu: "8"
```

```
    requests.memory: 12Gi
```

```
    limits.cpu: "16"
```

```
    limits.memory: 24Gi
```

```
    pods: "40"
```

```
    persistentvolumeclaims: "10"
```

```
kubectl apply -f k8s/namespaces/resource-quota.yaml
```

5.3 Common ConfigMap

```
# k8s/config/app-config.yaml
```

```
apiVersion: v1
```

```
kind: ConfigMap
```

```
metadata:
```

```
  name: app-config
```

```
  namespace: picloud doom
```

```
data:
```

```
  SPRING_PROFILES_ACTIVE: "prod"
```

```
  POSTGRES_HOST: "postgres"
```

```
  POSTGRES_PORT: "5432"
```

```
REDIS_HOST: "redis"
REDIS_PORT: "6379"
KAFKA_BOOTSTRAP_SERVERS: "kafka:29092"
KEYCLOAK_INTERNAL_URL: "http://keycloak:8080"
KEYCLOAK_EXTERNAL_URL: "http://auth.picloudboom.local"
KEYCLOAK_REALM: "myapp-realm"
MINIO_ENDPOINT: "http://minio:9000"
MINIO_PUBLIC_ENDPOINT: "http://minio.picloudboom.local"
AI_TRAINING_PATH_URL: "https://YOUR-AZURE-AI-APP.azurecontainerapps.io/predict-path"
KOKORO_BASE_URL: "https://YOUR-AZURE-KOKORO-APP.azurecontainerapps.io"
WHISPER_BASE_URL: "https://YOUR-AZURE-WHISPER-APP.azurecontainerapps.io"
kubectl apply -f k8s/config/app-config.yaml
```

5.4 Secrets

```
# k8s/secrets/app-secrets.yaml
apiVersion: v1
kind: Secret
metadata:
  name: app-secrets
  namespace: picloudboom
type: Opaque
stringData:
  POSTGRES_USER: "postgres"
  POSTGRES_PASSWORD: "devpassword"
  REDIS_PASSWORD: "devpassword"
  KEYCLOAK_ADMIN: "admin"
  KEYCLOAK_ADMIN_PASSWORD: "devpassword"
  MINIO_ACCESS_KEY: "minioadmin"
  MINIO_SECRET_KEY: "minioadmin"
  USER_SERVICE_CLIENT_SECRET: "change-me"
  INTERVIEW_SERVICE_CLIENT_SECRET: "change-me"
  TRAINING_SERVICE_CLIENT_SECRET: "change-me"
  GROQ_API_KEY: ""
  GEMINI_API_KEY: ""
  GOOGLE_AI_API_KEY: ""
  MAIL_USERNAME: ""
  MAIL_PASSWORD: ""
kubectl apply -f k8s/secrets/app-secrets.yaml
```

5.5 PostgreSQL with multiple databases

```
# k8s/infra/postgres/init-multiple-databases.sql
CREATE DATABASE userdb;
CREATE DATABASE interviewdb;
CREATE DATABASE trainingdb;
CREATE DATABASE quizdb;
CREATE DATABASE communitydb;
```

```

CREATE DATABASE resourcedb;
CREATE DATABASE mentorship_db;
CREATE DATABASE keycloakdb;
kubectrl create configmap postgres-init-sql \
  --from-file=k8s/infra/postgres/init-multiple-databases.sql \
  -n picloudoom \
  --dry-run=client -o yaml | kubectrl apply -f -
# k8s/infra/postgres/postgres.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-pvc
  namespace: picloudoom
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 15Gi
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres
  namespace: picloudoom
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:16-alpine
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRES_USER
              valueFrom:
                secretKeyRef:
                  name: app-secrets

```



```

    key: POSTGRES_USER
- name: POSTGRES_PASSWORD
  valueFrom:
    secretKeyRef:
      name: app-secrets
      key: POSTGRES_PASSWORD
  volumeMounts:
    - name: postgres-storage
      mountPath: /var/lib/postgresql/data
    - name: postgres-init
      mountPath: /docker-entrypoint-initdb.d/init-multiple-databases.sql
      subPath: init-multiple-databases.sql
  resources:
    requests:
      cpu: "500m"
      memory: "768Mi"
    limits:
      cpu: "1"
      memory: "1.5Gi"
  readinessProbe:
    exec:
      command: ["pg_isready", "-U", "postgres"]
    initialDelaySeconds: 15
    periodSeconds: 10
  volumes:
    - name: postgres-storage
      persistentVolumeClaim:
        claimName: postgres-pvc
    - name: postgres-init
      configMap:
        name: postgres-init-sql
---
apiVersion: v1
kind: Service
metadata:
  name: postgres
  namespace: picloudoom
spec:
  selector:
    app: postgres
  ports:
    - port: 5432
      targetPort: 5432
kubectl apply -f k8s/infra/postgres/postgres.yaml

```

5.6 Redis

```
# k8s/infra/redis/redis.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis
  namespace: picloudboom
spec:
  replicas: 1
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
        - name: redis
          image: redis:7-alpine
          ports:
            - containerPort: 6379
          command:
            - redis-server
            - --requirepass
            - $(REDIS_PASSWORD)
            - --maxmemory
            - 256mb
            - --maxmemory-policy
            - allkeys-lru
          env:
            - name: REDIS_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: app-secrets
                  key: REDIS_PASSWORD
      resources:
        requests:
          cpu: "100m"
          memory: "128Mi"
        limits:
          cpu: "500m"
          memory: "512Mi"
      readinessProbe:
        tcpSocket:
```

```

    port: 6379
    initialDelaySeconds: 5
    periodSeconds: 10
---
apiVersion: v1
kind: Service
metadata:
  name: redis
  namespace: picclouddoom
spec:
  selector:
    app: redis
  ports:
    - port: 6379
      targetPort: 6379
kubectl apply -f k8s/infra/redis/redis.yaml

```

5.7 Kafka and Zookeeper

Resource-limited choice

For the demo, use a single Kafka broker and single Zookeeper instance with replication factor 1. A production-grade Kafka cluster would require more resources.

```

# k8s/infra/kafka/kafka.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: zookeeper
  namespace: picclouddoom
spec:
  replicas: 1
  selector:
    matchLabels:
      app: zookeeper
  template:
    metadata:
      labels:
        app: zookeeper
    spec:
      containers:
        - name: zookeeper
          image: confluentinc/cp-zookeeper:7.5.0
          ports:
            - containerPort: 2181
          env:
            - name: ZOOKEEPER_CLIENT_PORT

```

```
    value: "2181"
  - name: ZOOKEEPER_TICK_TIME
    value: "2000"
  resources:
    requests:
      cpu: "250m"
      memory: "384Mi"
    limits:
      cpu: "750m"
      memory: "768Mi"
```

```
apiVersion: v1
kind: Service
metadata:
  name: zookeeper
  namespace: picloudoom
spec:
  selector:
    app: zookeeper
  ports:
    - port: 2181
      targetPort: 2181
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kafka
  namespace: picloudoom
spec:
  replicas: 1
  selector:
    matchLabels:
      app: kafka
  template:
    metadata:
      labels:
        app: kafka
    spec:
      containers:
        - name: kafka
          image: confluentinc/cp-kafka:7.5.0
          ports:
            - containerPort: 29092
          env:
            - name: KAFKA_BROKER_ID
```

```

    value: "1"
  - name: KAFKA_ZOOKEEPER_CONNECT
    value: "zookeeper:2181"
  - name: KAFKA_LISTENERS
    value: "INTERNAL://0.0.0.0:29092"
  - name: KAFKA_ADVERTISED_LISTENERS
    value: "INTERNAL://kafka:29092"
  - name: KAFKA_LISTENER_SECURITY_PROTOCOL_MAP
    value: "INTERNAL:PLAINTEXT"
  - name: KAFKA_INTER_BROKER_LISTENER_NAME
    value: "INTERNAL"
  - name: KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR
    value: "1"
  - name: KAFKA_AUTO_CREATE_TOPICS_ENABLE
    value: "false"
resources:
  requests:
    cpu: "500m"
    memory: "768Mi"
  limits:
    cpu: "1"
    memory: "1.5Gi"
---

```

```

apiVersion: v1
kind: Service
metadata:
  name: kafka
  namespace: picloudoom
spec:
  selector:
    app: kafka
  ports:
    - name: internal
      port: 29092
      targetPort: 29092

```

```
kubectl apply -f k8s/infra/kafka/kafka.yaml
```

5.8 Keycloak

```

# k8s/infra/keycloak/keycloak.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: keycloak
  namespace: picloudoom
spec:
  replicas: 1

```

```
selector:
  matchLabels:
    app: keycloak
template:
  metadata:
    labels:
      app: keycloak
  spec:
    containers:
      - name: keycloak
        image: quay.io/keycloak/keycloak:24.0.1
        args: ["start-dev"]
        ports:
          - containerPort: 8080
        env:
          - name: KC_DB
            value: postgres
          - name: KC_DB_URL
            value: jdbc:postgresql://postgres:5432/keycloakdb
          - name: KC_DB_USERNAME
            valueFrom:
              secretKeyRef:
                name: app-secrets
                key: POSTGRES_USER
          - name: KC_DB_PASSWORD
            valueFrom:
              secretKeyRef:
                name: app-secrets
                key: POSTGRES_PASSWORD
          - name: KEYCLOAK_ADMIN
            valueFrom:
              secretKeyRef:
                name: app-secrets
                key: KEYCLOAK_ADMIN
          - name: KEYCLOAK_ADMIN_PASSWORD
            valueFrom:
              secretKeyRef:
                name: app-secrets
                key: KEYCLOAK_ADMIN_PASSWORD
          - name: KC_HOSTNAME_STRICT
            value: "false"
          - name: KC_HTTP_ENABLED
            value: "true"
          - name: KC_PROXY
            value: "edge"
```

```

resources:
  requests:
    cpu: "300m"
    memory: "768Mi"
  limits:
    cpu: "1"
    memory: "1.5Gi"
readinessProbe:
  httpGet:
    path: /realms/master
    port: 8080
  initialDelaySeconds: 60
  periodSeconds: 10
---
apiVersion: v1
kind: Service
metadata:
  name: keycloak
  namespace: piclouddoom
spec:
  selector:
    app: keycloak
  ports:
    - port: 8080
      targetPort: 8080
kubectl apply -f k8s/infra/keycloak/keycloak.yaml

```

6. Backend microservices deployment

Service	Port	Database	Platform
user-service	8081	userdb	OpenStack Kubernetes
interview-service	8082	interviewdb	OpenStack Kubernetes
training-service	8083	trainingdb	OpenStack Kubernetes
mentorship-service	8084	mentorship_db	OpenStack Kubernetes
quiz-service	8085	quizdb	OpenStack Kubernetes
community-service	8086	communitydb	OpenStack Kubernetes
resource-service	8087	resourcedb	OpenStack Kubernetes
ai-training-path	8000	none	Azure Container Apps

Deployment rule

Deploy user-service first, validate it, then copy the same pattern for the other Spring Boot services. Do not apply all services at once.

```

# k8s/apps/user-service/user-service.yaml
apiVersion: apps/v1

```

```

kind: Deployment
metadata:
  name: user-service
  namespace: picclouddoom
spec:
  replicas: 1
  selector:
    matchLabels:
      app: user-service
  template:
    metadata:
      labels:
        app: user-service
    spec:
      containers:
        - name: user-service
          image: picclouddoomacr.azurecr.io/user-service:integration-final
          imagePullPolicy: Always
          ports:
            - containerPort: 8081
          env:
            - name: SERVER_PORT
              value: "8081"
            - name: SPRING_DATASOURCE_URL
              value: jdbc:postgresql://postgres:5432/userdb
            - name: SPRING_DATASOURCE_USERNAME
              valueFrom:
                secretKeyRef:
                  name: app-secrets
                  key: POSTGRES_USER
            - name: SPRING_DATASOURCE_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: app-secrets
                  key: POSTGRES_PASSWORD
            - name: SPRING_KAFKA_BOOTSTRAP_SERVERS
              valueFrom:
                configMapKeyRef:
                  name: app-config
                  key: KAFKA_BOOTSTRAP_SERVERS
            - name: SPRING_REDIS_HOST
              valueFrom:
                configMapKeyRef:
                  name: app-config
                  key: REDIS_HOST

```



```

- name: SPRING_REDIS_PASSWORD
  valueFrom:
    secretKeyRef:
      name: app-secrets
      key: REDIS_PASSWORD
- name: KEYCLOAK_SERVER_URL
  value: http://keycloak:8080
- name: KEYCLOAK_REALM
  value: myapp-realm
- name: KEYCLOAK_JWK_SET_URI
  value: http://keycloak:8080/realms/myapp-realm/protocol/openid-connect/certs
- name: AI_TRAINING_PATH_URL
  valueFrom:
    configMapKeyRef:
      name: app-config
      key: AI_TRAINING_PATH_URL
readinessProbe:
  httpGet:
    path: /actuator/health
    port: 8081
  initialDelaySeconds: 45
  periodSeconds: 10
livenessProbe:
  httpGet:
    path: /actuator/health
    port: 8081
  initialDelaySeconds: 90
  periodSeconds: 20
resources:
  requests:
    cpu: "250m"
    memory: "512Mi"
  limits:
    cpu: "750m"
    memory: "1Gi"
---
apiVersion: v1
kind: Service
metadata:
  name: user-service
  namespace: picloud doom
spec:
  selector:
    app: user-service
  ports:

```

```
- port: 8081
  targetPort: 8081
kubectl apply -f k8s/apps/user-service/user-service.yaml
kubectl get pods -n piclouddoom
kubectl logs -n piclouddoom deploy/user-service
```

7. Azure deployment for AI services

Azure is used for two roles: Azure Container Registry stores images, and Azure Container Apps runs heavy or variable-load AI services. This is the main hybrid deployment justification.

```
az login
az group create --name rg-piclouddoom --location francecentral
az acr create --resource-group rg-piclouddoom --name piclouddoomacr --sku Basic --admin-enabled true
az acr login --name piclouddoomacr
# Build and push images
docker build -t piclouddoomacr.azurecr.io/user-service:integration-final ./user-service
docker build -t piclouddoomacr.azurecr.io/interview-service:integration-final ./interview-service
docker build -t piclouddoomacr.azurecr.io/training-service:integration-final ./training-service
docker build -t piclouddoomacr.azurecr.io/quiz-service:integration-final ./quiz-service
docker build -t piclouddoomacr.azurecr.io/community-service:integration-final ./community-service
docker build -t piclouddoomacr.azurecr.io/resource-service:integration-final ./resource-service
docker build -t piclouddoomacr.azurecr.io/mentorship-service:integration-final ./mentorship-service
docker build -t piclouddoomacr.azurecr.io/ai-training-path:integration-final ./ai-training-path
docker push piclouddoomacr.azurecr.io/user-service:integration-final
docker push piclouddoomacr.azurecr.io/interview-service:integration-final
docker push piclouddoomacr.azurecr.io/training-service:integration-final
docker push piclouddoomacr.azurecr.io/quiz-service:integration-final
docker push piclouddoomacr.azurecr.io/community-service:integration-final
docker push piclouddoomacr.azurecr.io/resource-service:integration-final
docker push piclouddoomacr.azurecr.io/mentorship-service:integration-final
docker push piclouddoomacr.azurecr.io/ai-training-path:integration-final
# Azure Container Apps for ai-training-path
az containerapp env create \
  --name env-piclouddoom \
  --resource-group rg-piclouddoom \
  --location francecentral

az acr credential show --name piclouddoomacr

az containerapp create \
  --name ai-training-path \
  --resource-group rg-piclouddoom \
  --environment env-piclouddoom \
  --image piclouddoomacr.azurecr.io/ai-training-path:integration-final \
  --registry-server piclouddoomacr.azurecr.io \
```

```
--registry-username piclouddoomacr \
--registry-password YOUR_ACR_PASSWORD \
--target-port 8000 \
--ingress external \
--min-replicas 0 \
--max-replicas 2 \
--cpu 0.5 \
--memory 1Gi
```

```
az containerapp show \
--name ai-training-path \
--resource-group rg-piclouddoom \
--query properties.configuration.ingress.fqdn \
-o tsv
```

Azure justification

AI services are moved to Azure Container Apps because they can be resource-heavy and unevenly used. Azure Container Apps supports serverless container hosting, automatic scaling, revisions, and scaling rules, which protects OpenStack resources.

8. Vercel deployment for Angular frontend

The Angular frontend should be deployed on Vercel instead of OpenStack Kubernetes to avoid consuming cluster resources for static files.

```
cd frontend
npm install
npm run build
ls dist
# frontend/vercel.json
{
  "rewrites": [
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```

Vercel setting	Value
Git branch	deployment/hybrid-openstack-azure-vercel
Root Directory	frontend
Build Command	npm run build
Output Directory	dist/<app-name> or dist/<app-name>/browser depending on Angular version
Environment variables	API base URL, Keycloak URL, Azure AI URL

Vercel justification

The frontend is a static client-side application after production build. Vercel reduces OpenStack CPU/RAM usage and provides fast frontend deployment from GitHub.

9. Ingress and external access

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
```

```
helm repo update
```

```
helm install ingress-nginx ingress-nginx/ingress-nginx \
```

```
--namespace ingress-nginx \
```

```
--create-namespace
```

```
kubectl get pods -n ingress-nginx
```

```
kubectl get svc -n ingress-nginx
```

```
# k8s/ingress/picloudboom-ingress.yaml
```

```
apiVersion: networking.k8s.io/v1
```

```
kind: Ingress
```

```
metadata:
```

```
  name: picloudboom-ingress
```

```
  namespace: picloudboom
```

```
spec:
```

```
  ingressClassName: nginx
```

```
  rules:
```

```
    - host: api.picloudboom.local
```

```
      http:
```

```
        paths:
```

```
          - path: /api/users
```

```
            pathType: Prefix
```

```
            backend:
```

```
              service:
```

```
                name: user-service
```

```
              port:
```

```
                number: 8081
```

```
          - path: /api/interviews
```

```
            pathType: Prefix
```

```
            backend:
```

```
              service:
```

```
                name: interview-service
```

```
              port:
```

```
                number: 8082
```

```
          - path: /api/training
```

```
            pathType: Prefix
```

```
            backend:
```

```
              service:
```

```
                name: training-service
```

```

    port:
      number: 8083
- path: /api/mentorship
  pathType: Prefix
  backend:
    service:
      name: mentorship-service
    port:
      number: 8084
- path: /api/quizzes
  pathType: Prefix
  backend:
    service:
      name: quiz-service
    port:
      number: 8085
- path: /api/community
  pathType: Prefix
  backend:
    service:
      name: community-service
    port:
      number: 8086
- path: /api/resources
  pathType: Prefix
  backend:
    service:
      name: resource-service
    port:
      number: 8087
- host: auth.picclouddoom.local
http:
  paths:
    - path: /
      pathType: Prefix
      backend:
        service:
          name: keycloak
        port:
          number: 8080
kubectl apply -f k8s/ingress/picclouddoom-ingress.yaml
kubectl get ingress -n picclouddoom

```

10. CI/CD with GitHub Actions and Azure Container Registry

```
# .github/workflows/build-and-push-acr.yml
```

Pi-CloudDOOM Hybrid Deployment Guide - OpenStack Kubernetes + Azure + Vercel

name: Build and Push Images to ACR

on:

push:

branches:

- integration/final
- deployment/hybrid-openstack-azure-vercel

jobs:

build:

runs-on: ubuntu-latest

strategy:

matrix:

service:

- { name: user-service, path: ./user-service, image: user-service }
- { name: interview-service, path: ./interview-service, image: interview-service }
- { name: training-service, path: ./training-service, image: training-service }
- { name: quiz-service, path: ./quiz-service, image: quiz-service }
- { name: community-service, path: ./community-service, image: community-service }
- { name: resource-service, path: ./resource-service, image: resource-service }
- { name: mentorship-service, path: ./mentorship-service, image: mentorship-service }
- { name: ai-training-path, path: ./ai-training-path, image: ai-training-path }

steps:

- name: Checkout

uses: actions/checkout@v4

- name: Login to Azure Container Registry

uses: docker/login-action@v3

with:

registry: picloudoomacr.azurecr.io

username: \${ secrets.ACR_USERNAME }

password: \${ secrets.ACR_PASSWORD }

- name: Build Docker image

run: |

docker build \

-t picloudoomacr.azurecr.io/\${ matrix.service.image }:\${ github.sha } \

-t picloudoomacr.azurecr.io/\${ matrix.service.image }:integration-final \

\${ matrix.service.path }

- name: Push Docker image

run: |

docker push picloudoomacr.azurecr.io/\${ matrix.service.image }:\${ github.sha }

docker push picloudoomacr.azurecr.io/\${ matrix.service.image }:integration-final

11. Observability, HPA, and resource optimization

helm repo add prometheus-community <https://prometheus-community.github.io/helm-charts>

helm repo update

```

helm install monitoring prometheus-community/kube-prometheus-stack \
--namespace monitoring \
--create-namespace \
--set prometheus.prometheusSpec.resources.requests.cpu=200m \
--set prometheus.prometheusSpec.resources.requests.memory=512Mi \
--set prometheus.prometheusSpec.resources.limits.cpu=500m \
--set prometheus.prometheusSpec.resources.limits.memory=1Gi \
--set grafana.resources.requests.cpu=100m \
--set grafana.resources.requests.memory=256Mi \
--set grafana.resources.limits.cpu=300m \
--set grafana.resources.limits.memory=512Mi
kubectl get pods -n monitoring
kubectl port-forward -n monitoring svc/monitoring-grafana 3000:80
# Example HPA for user-service
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: user-service-hpa
  namespace: picloudboom
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: user-service
  minReplicas: 1
  maxReplicas: 3
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70

```

12. Final validation checklist and screenshots

Screenshot / proof	Command or location	Grid criterion
Dockerfiles per service	Repository folders: user-service, interview-service, etc.	Containerization
Images in ACR	az acr repository list --name picloudboomacr -o table	Docker images on registry
Kubernetes nodes Ready	kubectl get nodes -o wide	Infrastructure access
Pods Running	kubectl get pods -n picloudboom -o wide	Service deployment
Services created	kubectl get svc -n picloudboom	Inter-service communication
Ingress created	kubectl get ingress -n	Application exposure

	piclouddoom	
ConfigMaps and Secrets	kubectl get configmap,secret -n piclouddoom	Configuration management
Resource limits	kubectl describe deployment user-service -n piclouddoom	Resource optimization bonus
HPA	kubectl get hpa -n piclouddoom	Scalability
Grafana dashboard	http://localhost:3000 after port-forward	Observability bonus
Vercel frontend	Vercel dashboard + deployed URL	Hybrid deployment
Azure Container App	az containerapp show ...	Hybrid deployment

```
# Final validation commands
kubectl get nodes -o wide
kubectl get pods -n piclouddoom -o wide
kubectl get svc -n piclouddoom
kubectl get ingress -n piclouddoom
kubectl get configmap -n piclouddoom
kubectl get secret -n piclouddoom
kubectl get hpa -n piclouddoom
kubectl logs -n piclouddoom deploy/user-service
kubectl run curl-test -n piclouddoom --rm -it --image=curlimages/curl -- sh
```

13. Troubleshooting guide

Problem	Likely cause	Fix
Pod stuck Pending	No resources or no storage class	Check kubectl describe pod and kubectl get storageclass. Reduce limits or install local-path provisioner.
ImagePullBackOff	Cluster cannot pull from ACR	Create an imagePullSecret or make sure registry credentials are configured.
CrashLoopBackOff	Bad env var, DB not ready, wrong port	Check kubectl logs and compare Spring Boot config with docker-compose.yml.
Backend cannot reach Keycloak	Using localhost inside pod	Use http://keycloak:8080 internally and auth.piclouddoom.local for browser access.
Frontend login redirect fails	Wrong Keycloak public URL	Update Keycloak client redirect URIs for the Vercel domain.
Kafka connection fails	Wrong bootstrap server	Use kafka:29092 inside Kubernetes.
Postgres DB missing	Init SQL did not run because PVC already existed	Delete PVC only in demo if data can be lost, then redeploy PostgreSQL.
Ingress not accessible	No LoadBalancer IP / hosts not	Use NodePort, OpenStack load

	configured	balancer, or /etc/hosts mapping.
--	------------	----------------------------------

14. Report-ready justification text

Hybrid deployment justification

Due to limited resources in the OpenStack infrastructure, we adopted a hybrid deployment strategy. The core backend microservices, databases, authentication server, message broker, object storage, and monitoring stack were deployed on the OpenStack Kubernetes cluster. The Angular frontend was deployed on Vercel because it is a static production build and does not require Kubernetes resources. The AI-related service was deployed on Azure Container Apps to benefit from automatic scaling and reduced resource consumption on OpenStack. This architecture preserves the required Kubernetes deployment while optimizing infrastructure usage and improving maintainability.

Resource optimization justification

Resource requests and limits were added to all Kubernetes workloads to avoid overloading the limited OpenStack cluster. Horizontal Pod Autoscaling was enabled only for critical services, with conservative max replicas, so the system can scale without exhausting the infrastructure.

Security/configuration justification

Non-sensitive configuration is stored in ConfigMaps, while credentials and API keys are injected through Kubernetes Secrets. This separates application code from environment-specific configuration and follows Kubernetes deployment best practices.

Sources and official documentation

The following official sources were used to justify the recommended platform choices:

- Azure Container Apps overview: <https://learn.microsoft.com/en-us/azure/container-apps/overview>
- Azure Container Apps scaling: <https://learn.microsoft.com/en-us/azure/container-apps/scale-app>
- Azure Container Registry documentation: <https://learn.microsoft.com/en-us/azure/container-registry/>
- Angular deployment documentation: <https://angular.dev/tools/cli/deployment>
- Vercel build configuration: <https://vercel.com/docs/builds/configure-a-build>
- Vercel Angular deployment guide: <https://vercel.com/kb/guide/deploying-angular-with-vercel>
- Argo project / GitOps tools: <https://argoproj.github.io/>